

Experiment No.	2
Experiment Title	Email Spam/Ham Classification using Naive Bayes and KNN ¹
Student Name	Sanjay S
Register Number	3122247001056
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	03/08/2026

1 Objective

Build spam classifiers with Naive Bayes and KNN, compare Gaussian, Multinomial and Bernoulli NB, study how accuracy changes with k , compare KDTree against BallTree, tune KNN with GridSearchCV and RandomizedSearchCV, and check speed and cross validated accuracy across all of them.

2 Problem Statement

Given Spambase, 4601 emails described by 57 numeric features (word/character frequencies and capital letter run stats), predict spam or ham, and compare how Naive Bayes and KNN behave on the same data, including speed.

3 Dataset Description

Dataset Name	Spambase
Dataset Source	Kaggle, somesh24/spambase (originally UCI ML Repository)
Number of Samples	4601
Number of Features	57
Number of Classes	2 (spam = 1, ham = 0)
Missing Values	none
Train-Test Split	80/20, stratified, plus 5 fold cross validation

4 Exploratory Data Analysis

- Class distribution: 2788 ham vs 1813 spam, mildly imbalanced.
- Correlation and feature distribution: heavily right skewed word frequencies, computed for the ten features most correlated with the label.
- No missing values to handle.

¹GitHub Repository: <https://github.com/sanjay-6727/Introdution-to-Machine-Learning->



Figure 1: Class distribution, full correlation heatmap, and histogram, box plot, violin, scaling, outlier, pair plot and Q-Q panels for the ten most predictive features.

Inference:

word_freq_your, word_freq_000, word_freq_remove and the exclamation mark frequency correlate most strongly with spam, matching common sense about spam wording. Most features spike at zero with a long tail, since a given email only uses a handful of the tracked words. The pair plot already shows some visible separation between spam and ham, a good early sign for a simple classifier.

5 Data Preprocessing

- No missing values, no encoding needed, all 57 features are numeric.
- Gaussian NB and KNN trained on StandardScaler output. Multinomial and Bernoulli NB trained on the raw counts, since both expect non-negative input and scaling would break that.
- 80/20 stratified split, 5 fold cross validation used throughout tuning.

Listing 1: Naive Bayes variants: Gaussian on scaled data, Multinomial/Bernoulli on raw counts

```

1 gnb = GaussianNB().fit(X_train_scaled, y_train)
2 mnb = MultinomialNB().fit(X_train_raw, y_train)
3 bnb = BernoulliNB().fit(X_train_raw, y_train)
4
5 grid = GridSearchCV(KNeighborsClassifier(), param_grid,
6                     cv=5, scoring="accuracy", n_jobs=-1)

```

```

7 | grid.fit(X_train_scaled, y_train)
8 | print(grid.best_params_, grid.best_score_)

```

6 Machine Learning Algorithm

Naive Bayes applies Bayes' theorem with a conditional independence assumption: $P(C | X) = \frac{P(X|C)P(C)}{P(X)}$. Gaussian NB assumes normal features, Multinomial models counts, Bernoulli treats features as presence/absence. All three train fast but the independence assumption costs accuracy when features correlate.

K-Nearest Neighbors stores the training set and votes among the k closest points at prediction time. No assumption about boundary shape, but every prediction costs a distance search, so it is the more expensive of the two at inference time.

7 Experimental Setup

Python Version	3.11.9
Scikit-Learn Version	1.7.2
IDE	Anaconda Navigator / VS Code
Operating System	Windows 11
Hardware	

8 Hyperparameter Selection

Parameter	Value
KNN k values tried	1, 3, 5, 7, 9, 11
GridSearchCV grid	n_neighbors in {1,3,5,7,9,11,15,21}, weights, metric, algorithm
GridSearchCV best	k=11, manhattan, distance, kd_tree, CV accuracy 0.9228
RandomizedSearchCV best	k=9, manhattan, distance, kd_tree, CV accuracy 0.9212

9 Results

Naive Bayes comparison

Metric	Gaussian	Multinomial	Bernoulli
Accuracy	0.8328	0.7763	0.8762
Precision	0.7146	0.7199	0.8716
Recall	0.9587	0.7080	0.8044
F1	0.8188	0.7139	0.8367
ROC-AUC	0.9376	0.8248	0.9496

KNN across k

k	Accuracy	Precision	Recall	F1
1	0.8990	0.8792	0.8623	0.8707
5	0.9077	0.8861	0.8788	0.8824
9	0.9088	0.8930	0.8733	0.8830
11	0.9099	0.9000	0.8678	0.8836

KDTree vs BallTree (k=11)

Metric	KDTree	BallTree
Accuracy	0.9099	0.9099
Training Time (s)	0.0266	0.0180
Prediction Time (s)	0.2015	0.1827

Measured training / prediction time

Algorithm	Train (s)	Predict (s)
Gaussian NB	0.00398	0.00079
Bernoulli NB	0.00548	0.00172
Best KNN (tuned)	0.02411	0.19402

10 Result Visualization

Experiment_2 - results (14 images)

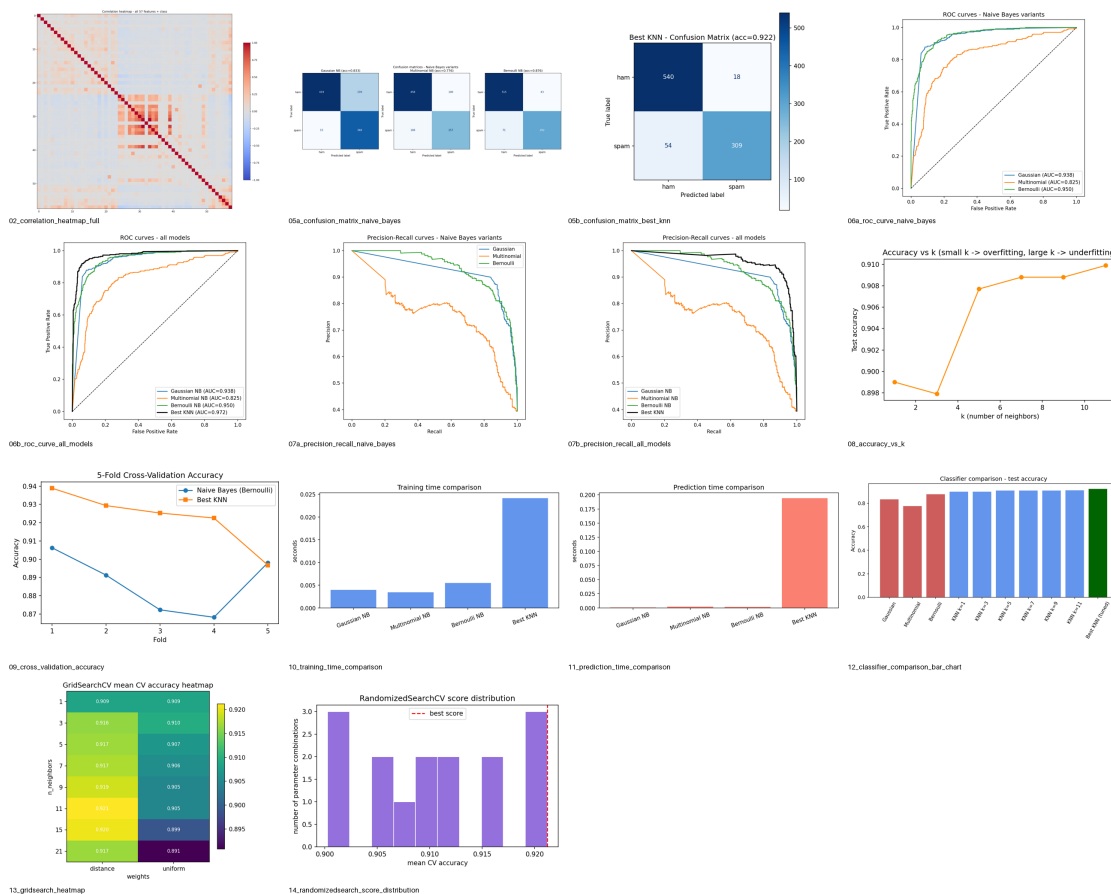


Figure 2: Confusion matrices, ROC and precision-recall curves, accuracy vs k, cross validation accuracy, timing bars, classifier comparison, GridSearchCV heatmap and RandomizedSearchCV score distribution.

Inference:

Bernoulli NB wins the three way comparison on accuracy, precision and ROC-AUC, Spambase behaves more like presence/absence data than raw counts. Gaussian NB has the best recall but a lot of false positives. Accuracy vs k rises to about k=9-11 then flattens, small k overfits noisy neighbors, large k underfits. GridSearchCV and RandomizedSearchCV landed on almost the same accuracy despite the randomized search checking far fewer combinations and finishing about eight times faster. KDTree and BallTree give identical accuracy as expected, only speed differs. The timing bars make the core tradeoff obvious, Naive Bayes runs in milliseconds regardless of test size, KNN's prediction time is the most expensive number on the chart.

11 Discussion

Naive Bayes' independence assumption clearly costs accuracy compared to tuned KNN (0.8872 vs 0.9226 average CV accuracy), but it wins easily on speed, a gap that only grows with more data. Bernoulli NB beating the other two variants fits Spambase being

closer to a presence/absence signal than a count signal. KNN's prediction time already looks expensive at only 4601 rows, this would need an approximate search method to scale further.

12 Conclusion

Tuned KNN beats every Naive Bayes variant on accuracy, but Naive Bayes is far cheaper to run. For a high volume mail filter, that tradeoff could easily flip the choice toward Naive Bayes. GridSearchCV and RandomizedSearchCV agreeing closely gives confidence that $k=11$, manhattan distance and distance weighting is a genuinely good setting here, not a search artifact.

13 Learning Outcomes

- Saw firsthand why the independence assumption in Naive Bayes trades away some accuracy for a big speed advantage.
- Understood why KNN needs scaled features and why its prediction cost, not training cost, is the real bottleneck.
- Compared GridSearchCV and RandomizedSearchCV directly and saw the randomized version get almost the same answer much faster.
- Learned that KDTree and BallTree only change search speed, not the actual predictions.

14 References

1. Stephen Marsland, "Machine Learning - An Algorithmic Perspective", 2nd Edition, Chapman and Hall/CRC Machine Learning and Pattern Recognition Series, 2015.
2. Ethem Alpaydin, "Introduction to Machine Learning", 3rd Edition, The MIT Press, 2014.